PBL Milestone 1: Algorithm Architecture Specification & Software Engineering Proposal

ALGOCOM (Algorithms and Complexity) / SOFTENG (Software Engineering) · Team Assignment

Metadata Field	Project Team Specification
Project Title	MetroFlow
Team Name	F4
Section & Year	III-CCSAD
Team Members & Roles	1. Lead Algorithm Architect: Lapating, Khent U. 2. Backend / Systems Engineer: De Vera, Josiah Ray S. 3. Data / QA Engineer: Vergara, John Gabriel D. 4. Systems Analyst / Technical Lead: Felipe, Marck Aaron V.
Repository URL	https://github.com/siahray/MetroFlow.git
Submission Date	November 4, 2026

Proposal at a glance

Computational problem. MetroFlow is a flood-response and rescue-dispatch system. Its core computation is a *vehicle-specific, flood-aware shortest-time path* on a live road graph: given a rescue vehicle's ground clearance and the current standing-water depth on every road segment, find the fastest route that never enters a segment the vehicle cannot cross.

Core algorithm. A* search with a min-heap frontier, an admissible and consistent great-circle/top-speed heuristic, a hard clearance cut-off, and a depth-dependent speed-degradation edge cost [1], [2]. This is implemented in `backend/routing.py` (`td_a_star_search`).

Upstream input generator. A hydrology engine (SCS Curve Number runoff → ponding depth) converts rainfall into the per-segment depths the router consumes [8].

Contents. Task 1 Problem Modeling · Task 2 Paradigm & Pseudocode · Task 3 Complexity & Scalability · Task 4 Decoupled Architecture & API Schema · Task 5 Agile Backlog & Benchmarks · Task 6 Defense Questions · References · Presentation Deck (12 slides, appended).

> <!-- AI Assistance: We used Claude (Claude Code) to read our existing repository, draft the mathematical formulation, complexity derivations, pseudocode, JSON schemas and review answers in this document, and to lay out the report. The team reviewed each formula against `backend/routing.py` and `backend/hydrology.py`. -->

Task 1: Computational Problem Modeling & Formal Mathematical Formulation

1.1 Problem statement & domain context

During typhoon-driven flooding, standing water makes road segments impassable for some vehicles and merely slow for others. A rescue unit with 28 cm of ground clearance cannot use a lane that a 70 cm truck can. Routing that ignores depth and vehicle clearance sends units into submerged streets. The operational question is therefore: *for a given vehicle, what is the fastest route to a victim that only uses road segments the vehicle can actually cross, and does any such route exist?*

Flood depth is a standard predictor of road-network disruption: speed falls as depth rises and vehicles stop at a depth tied to their clearance [7]. Prior work on emergency services during floods assigns depths to road links and derives reduced vehicle speeds and service areas from them [6]. MetroFlow adopts the same modelling idea, using a per-vehicle clearance threshold and a depth-dependent speed penalty, and runs it on a live, sensor- and report-updated graph. Shortest-path search under constraints is a core graph problem [1], and goal-directed A* search is the standard way to cut the explored region on road networks [2], [5].

1.2 Formal specification

Input parameters (I). A directed graph $G = (V, E)$ with $|V| = n$, $|E| = m$. Each node $u \in V$ has a coordinate $x_u = (\varphi_u, \lambda_u)$ (latitude, longitude). Each edge $e = (u, v) \in E$ has length $\ell_e > 0$ (m), posted speed $s_e > 0$ (km/h) and live water depth $d_e \geq 0$ (cm). A query supplies a source $s \in V$, target $t \in V$ and vehicle clearance $c > 0$ (cm).

Depths d_e are produced by the hydrology engine: for rainfall P (mm), curve number CN, $S = \frac{25400}{CN} - 254$ and initial abstraction $I_a = 0.2S$, the direct runoff is

$$Q = \begin{cases} 0 & P \leq I_a \\ \dfrac{(P - I_a)^2}{P - I_a + S} & P > I_a \end{cases}$$

which is then scaled to a street ponding depth in cm [8]. Verified hazards and sensor breaches can additionally close edges.

Boundary conditions & constraints (C).

- Passability: $d_e < c$ (an edge with $d_e \geq c$ is *submerged* and removed). The feasible subgraph is $G_c = (V, E_c)$, \(E_c=\{e: d_e
- $0 < c \leq 200$ cm (enforced at the API); $s, t \in V$.
- Effective speed floor: $v_e \geq 0.5$ m/s so every traversal time is finite.
- Depth and speed are in cm and km/h throughout (project unit policy).

Objective function (f). This is an **optimization problem** (minimization) over s–t paths in G_c. Define the depth-degraded speed and traversal time of an edge:

$$v_e = \max\!\left(\frac{s_e/3.6}{1 + (d_e/10)^{1.8}},\ 0.5\right)\ [\mathrm{m/s}], \qquad w_e = \frac{\ell_e}{v_e}\ [\mathrm{s}]$$

$$\min_{P=(e_1,\ldots,e_k)\ \text{path } s\to t \text{ in } G_c} T(P) = \sum_{i=1}^{k} w_{e_i}$$

A companion **decision problem** is: does an s–t path exist in G_c? This is the 404 / "no safe route" case.

Output certificate (O). A deterministic tuple $(\text{status},\ P^*,\ T^*)$: `status` ∈ {OK, NO_SAFE_ROUTE, INVALID_NODE}; P^* the node sequence; $T^* = T(P^*)$ the minimal travel time. If no path exists, $P^* = \emptyset$ and $T^* = \infty$. Optimality is checkable: T^* must equal the Dijkstra reference value on the same G_c [4].

Dispatch extension. For an SOS with K candidate vehicles, each with clearance c_k and position s_k, the preview solves K independent instances and ranks units by T_k^*.

<!-- AI Assistance: Claude helped formalise the objective function and the SCS-CN statement from our code. The speed-degradation exponent 1.8 and the 0.5 m/s floor come from our existing implementation. They are modelling choices of the team, not values taken from [6] or [7]. -->

Task 2: Algorithmic Paradigm Selection & Logic Specification

Schema Field	Team Engineering Details
Algorithm Name	Clearance-Constrained, Depth-Penalized A* (implemented as `td_a_star_search`; "time-dependent" in the sense that edge costs are recomputed from the live depth state on every query, not that cost varies during one search).
Design Paradigm	Graph Algorithms & Network Theory: best-first search with an admissible, consistent heuristic over a min-heap priority queue.
Foundational References	Hart, Nilsson & Raphael (A*) [2]; Dechter & Pearl (optimality of A*) [3]; Dijkstra [4]; Cormen et al., Ch. 22 and 24 [1]; road-network context in Bast et al. [5].
Baseline Alternative	Exhaustive simple-path enumeration (DFS with no pruning), then take the minimum. Secondary baseline: Dijkstra ($h \equiv 0$), which is correct but explores in all directions.
Computational Justification	The number of simple s–t paths grows factorially in n, so enumeration is infeasible beyond toy graphs. A* returns the same optimum in $O(m \log n)$ and, with a consistent heuristic, settles each node at most once; the heuristic additionally steers the frontier toward t, expanding fewer nodes than Dijkstra [1], [2], [3].
Invariants & Recurrence Relations	Bellman recurrence on the feasible graph: $g(s) = 0, \quad g(v) = \min_{(u,v)\in E_c} \big(g(u) + w_{uv}\big)$. Priority: $f(v) = g(v) + h(v)$. Heuristic: $h(v) = \dfrac{\text{hav}(x_v, x_t)}{v_{\max}}$, $v_{\max} = \max_e s_e/3.6$. **Admissible:** $w_{uv} \geq \ell_{uv}/v_{\max} \geq \text{hav}(x_u, x_v)/v_{\max}$ (the penalty factor is ≥ 1), so h never overestimates the remaining time. **Consistent:** $h(u) \leq w_{uv} + h(v)$ by the triangle inequality of great-circle distance, so f is non-decreasing along any path and the first time a node is popped its g is optimal [2], [3]. Loop invariant: every node in `settled` has final optimal g; every heap entry has $g \geq g_{\text{best}}$.

Structured pseudocode

```
TD-ASTAR(G, coords, s, t, c):                  // G: adjacency u -> [(v, len_m, speed_kmh,
depth_cm)]
 1  if s not in coords or t not in coords: return (INVALID_NODE, [], inf)
 2  if s == t: return (OK, [s], 0)
 3  v_max  ← max(max over all edges of speed_kmh / 3.6, 0.1)          // m/s, fixed for this
query
 4  h(x)   ← haversine(coords[x], coords[t]) / v_max
 5  g[x]   ← inf for all x;  g[s] ← 0;  parent ← {};  settled ← {}
 6  PQ     ← min-heap { (h(s), 0, s) }                                  // (f, g, node)
 7  while PQ not empty:
 8      (f, gu, u) ← POP-MIN(PQ)
 9      if u in settled or gu > g[u]: continue                          // stale duplicate
10      settled ← settled ∪ {u}
11      if u == t: return (OK, RECONSTRUCT(parent, s, t), g[t])
12      for (v, len, speed, depth) in G[u]:
13          if depth ≥ c: continue                                        // hard clearance
cut-off
14          pen ← 1 + (depth / 10)^1.8                                    // depth penalty,
pen ≥ 1
15          v_eff ← max((speed / 3.6) / pen, 0.5)                         // m/s, floor keeps
cost finite
16          g' ← gu + len / v_eff
17          if g' < g[v]: g[v] ← g'; parent[v] ← u; PUSH(PQ, (g' + h(v), g', v))
18  return (NO_SAFE_ROUTE, [], inf)                                          // frontier
exhausted: t unreachable in G_c

RECONSTRUCT(parent, s, t): follow parent[] from t back to s; reverse.
```

Termination: each node is settled at most once and edges are finite, so the loop ends in at most $m + 1$ pops; line 18 is the termination certificate for the infeasible case.

> <!-- AI Assistance: Claude produced the pseudocode by reading `td_a_star_search` and wrote the admissibility/consistency argument. The team should be able to re-derive both on the board. Note: an earlier version of our code used a hard-coded 50 km/h in h and was inadmissible; our regression fuzz test in `tests/test_routing.py` guards that fix. -->

Task 3: Theoretical Asymptotic Analysis & Scalability Profiling

3.1 Worst-case time $O(\cdot)$

1. **Initialisation** (lines 3–6): scanning all edges for $v_{\max}$ is $O(m)$; setting $g[x] = \infty$ for all nodes is $O(n)$. Subtotal $O(n + m)$.
2. **Settlement bound.** Because h is consistent (Task 2), a node's first valid pop carries its optimal g and it is never re-expanded [3]. Hence lines 10–17 run for each node at most once.
3. **Relaxations.** Line 12 iterates the out-edges of the settled node, so across the whole run each edge is examined at most once: at most m executions of lines 13–17, each $O(1)$ apart from the push.
4. **Heap operations.** Each successful relaxation pushes one entry, so at most $m + 1$ entries are ever pushed and popped. Each push or pop costs $O(\log |PQ|) \le O(\log(m + 1))$. Since $m \le n^2$, $\log m \le 2 \log n$, i.e. $O(\log n)$ [1].
5. **Total.** $T_{\text{worst}}(n, m) = O(n + m) + O\big((m + 1) \log n\big) = O(m \log n)$ for a connected road graph ($m \ge n - 1$). With $h \equiv 0$ the same bound is Dijkstra's with a binary heap [1], [4]; A* never does worse.

The master theorem does not apply: the algorithm is iterative, with no divide-and-conquer recurrence [1].

Baseline: exhaustive simple-path enumeration visits up to $(n-2)!$ paths on a dense graph, i.e. $O(n!)$ [1].

3.2 Best- and average-case time $\Omega(\cdot)$, $E[\cdot]$

Best case. If h is near-perfect (a near-straight, uniform-speed corridor), the search only settles nodes on the path of length k: $\Omega(k \log n)$ plus the $O(n+m)$ initialisation done by the current code.

Average case. City road graphs are sparse ($m = \Theta(n)$, degree about 3–4) and near-planar. A goal-directed search settles only a region around the straight line $s \to t$, typically far fewer than n nodes. This is expected behaviour [5] and is what Task 5's benchmark measures; we do not claim a closed-form bound for it. Heavy flooding lowers v_e while $v_{\max}$ is unchanged, which weakens h and moves the average case toward Dijkstra's.

3.3 Auxiliary space $S(\cdot)$

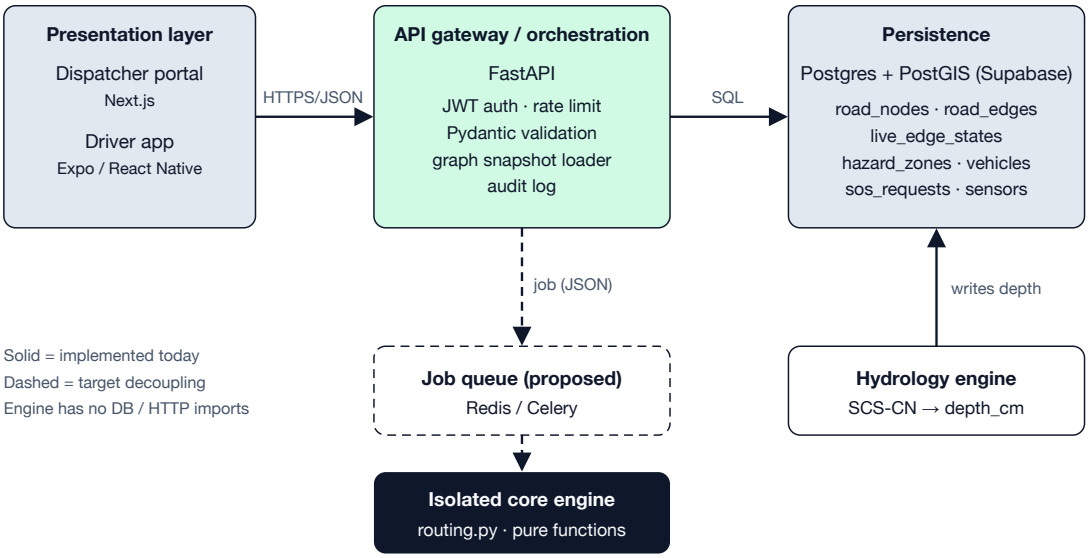
`g`, `parent`, `settled`: $O(n)$ each. Heap: lazy deletion allows stale duplicates, so up to $m+1$ entries. No recursion (iterative loop, path rebuilt iteratively). Total $S = O(n+m)$. The graph itself is $O(n+m)$ input, not auxiliary.

3.4 Big Data (3 V's) evaluation

Dimension	Engineering Parameter	System Scalability Impact & Mitigation Strategy
Volume	Pilot: a seeded district graph (`backend/network_seed.py`). Projected: a metro-scale graph, order 10^5 nodes and 4×10^5 directed edges (a planning figure, to be replaced by the real extract).	Python dict adjacency at that size is tens to low hundreds of MB. Mitigation: cache the static topology in the worker and refresh only the live depth overlay; load by bounding box (PostGIS `ST_DWithin` [14]) around s and t instead of the whole city; use compact arrays (CSR) if memory binds.
Velocity	Burst arrival during a typhoon: an SOS triggers K vehicle previews (one search each). Target: a few to tens of queries/s, with depth updates arriving from sensors and reports.	Searches are CPU-bound and independent. Run them in a worker pool (async queue) so request threads stay free; return a job id when a batch would exceed the request time budget. Searches read an immutable snapshot of the depth state, so concurrent writers cannot change costs mid-query.
Variety	Heterogeneous inputs: GeoJSON/PostGIS geometry, sensor readings, citizen reports, hazard polygons, rainfall values.	Normalise at the gateway into the one in-engine form (`depth_cm`, `speed_kmh`, `length_m`), validate with Pydantic, and keep unit conversion in a single place. The engine sees only numbers and node ids.

Task 4: Decoupled Software Architecture & Service-Layer Integration

4.1 Component diagram



Current state vs. target. Today `routing.py` is already a pure module: it imports no ORM, HTTP or database code and takes plain dicts and numbers. FastAPI endpoints (`main.py`) load the graph with SQLAlchemy, call it in-process, and format the response. The **target** shown by the dashed boxes moves the call behind a job queue and a worker, so the engine can be scaled and timed out independently of web threads. Boundaries: Presentation never reaches the database; the gateway owns auth, validation and persistence; the engine owns computation only [12].

4.2 Interface data contract

Request: `POST /api/v1/algorithm/solve` (JSON, `application/json`, JWT bearer; documented as OpenAPI 3.1 [11]). It generalises the existing `GET /api/v1/routing/navigate` and `GET /api/v1/dispatch/{id}/preview`.

```
{
  "request_id": "7c1e2f0a-3b8e-4a1c-9d57-0a1f2b3c4d5e",
  "algorithm": "td_a_star",
  "start_node": 101,
  "target_node": 482,
  "vehicle": { "vehicle_id": 7, "clearance_cm": 28.0 },
  "options": { "ignore_hazards": false, "time_limit_ms": 250 }
}
```

Constraints: `start_node`, `target_node` integer ≥ 1; `clearance_cm` number, 0 < x ≤ 200; `time_limit_ms` integer 1–5000; unknown fields rejected.

Response certificate (`200 OK`; asynchronous mode returns `202` with `job_id` and the same body on `GET /api/v1/algorithm/jobs/{job_id}`):

```
{
  "request_id": "7c1e2f0a-3b8e-4a1c-9d57-0a1f2b3c4d5e",
  "status": "OK",                       // OK | NO_SAFE_ROUTE | INVALID_NODE | TIMEOUT
  "result": {
    "path": [101, 134, 219, 482],
    "travel_time_s": 412.6,
    "distance_m": 3180.4,
    "segments": [ { "from": 101, "to": 134, "length_m": 820.0, "depth_cm": 6.0,
"passability": "LIGHT_FLOOD" } ],
    "hazards_on_route": []
  },
  "metrics": { "nodes_settled": 348, "edges_relaxed": 1210, "runtime_ms": 14.2, "optimal":
true },
  "flags": { "used_ignore_hazards": false, "truncated": false }
}
```

Errors use RFC 9457 `application/problem+json` [9] with HTTP semantics per RFC 9110 [10]: `422` invalid body, `404` / status `NO_SAFE_ROUTE` when no passable route exists, `401/403` auth, `429` rate limit, `504` `TIMEOUT`.

Task 5: Agile Implementation Backlog & Empirical Validation Plan

Process note. MetroFlow follows a waterfall lifecycle (BRD → FSD → TSD → implementation → verification). The user stories and ticket below use the format this milestone requires, but they are *not* sprint items: each is a work-package statement traced to an existing requirement. Story 1 traces to FSD-FR-089 (rescue route preview, per-vehicle) and TSD ADR-004 (A* routing); Story 2 to FSD-FR-057 (turn-by-turn routing) and the hazard-blocking rule in TSD §2.6. DEV-ALGO-101 is verified in the TSD's test-case phase, not a sprint review.

5.1 User stories

1. **As a** dispatcher, **I want** the system to run the flood-aware A* search from every available rescue vehicle to a victim's SOS location using each vehicle's own clearance, **so that** I can assign the unit with the lowest safe travel time and never send a unit into water it cannot cross.
2. **As a** rescue driver, **I want** the route calculation to re-run on the latest water depths and hazards, and tell me clearly when no safe route exists, **so that** I am rerouted around newly flooded streets instead of being routed into one.

5.2 Implementation & benchmarking ticket

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-101
Module / Service	`backend/routing.py` (engine), `backend/tests/test_routing.py` and a new `backend/tests/test_routing_benchmark.py` (benchmark); API wiring in `backend/main.py`.
Associated Algorithm	Clearance-constrained, depth-penalized A* with min-heap (`td_a_star_search`).
Edge-Case & Security Defenses	• Unknown `start`/`target` node → `INVALID_NODE`, not an exception. • `start == target` → `[start]`, $T = 0$. • Empty graph, or disconnected components (target unreachable) → `NO_SAFE_ROUTE`, `inf`. • Depth $\geq$ clearance on every cut edge → no route (404). Depth exactly equal to

	clearance is blocked. • Zero or negative speed, NaN / negative depth → reject at the gateway; speed floor 0.5 m/s in the engine prevents division by zero. • Clearance bound \(0 • Rate limiting and JWT auth at the gateway; per-query time limit to stop abusive or pathological inputs.
Concrete Implementation Steps	1. Build the adjacency map from `road_edges` joined with `live_edge_states` (length, speed, depth), excluding hazard-closed edges. 2. Compute $v_{\max}$ and the haversine heuristic for the query target. 3. Initialise g, `parent`, `settled` and the min-heap with $(h(s), 0, s)$. 4. Pop, skip stale/settled entries, settle, and return on the target. 5. Relax out-edges: apply the clearance cut-off, depth penalty and speed floor; push improved neighbours. 6. Reconstruct the path; return the certificate with metrics (nodes settled, runtime). 7. Add a Dijkstra reference implementation in tests for the optimality check. 8. Add the benchmark test and wire it into CI.
Automated Benchmarking Assertion	• `assert astar_cost == pytest.approx(dijkstra_cost)` on 10,000 random graphs/queries (optimality; extends the existing fuzz guard). • `assert p95_runtime_ms < 250` on a synthetic grid graph with N = 5,000 nodes (about 20,000 edges), 100 random queries. • `assert nodes_settled_astar <= nodes_settled_dijkstra` on the same queries (heuristic never hurts). • `assert route is None` (empty path, `inf`) when every edge on the only path has depth ≥ clearance.

The 250 ms / N = 5,000 threshold is a target we set, not yet a measured result. The CI job will fail the build if it is exceeded, and we will record the measured value in the repository.

Task 6: Academic Review Questions & Defense Checkpoints

Q1. Why must heavy algorithmic processing be decoupled from database ORMs and web request threads?

(a) **Responsiveness:** a CPU-bound search holds a worker thread for its whole run; with a bounded thread pool, K concurrent searches starve unrelated requests (login, map reads) [12]. Moving them to a worker pool behind a queue [15] bounds request latency and allows independent scaling. (b) **Testability and correctness:** a pure function over plain numbers can be unit-tested and fuzzed against a reference (Dijkstra) without a database; ORM lazy loading inside the loop would add a hidden query per edge, turning $O(m \log n)$ compute into $O(m)$ database round-trips. (c) **Consistency:** the engine should read an immutable snapshot so concurrent depth writes cannot change costs mid-search, which would break the optimality argument. (d) **Replaceability:** the same engine can be called by the dispatch preview, the driver app and batch jobs.

Q2. If the algorithm has exponential worst-case complexity, what keeps it viable?

Our core engine is not exponential: it is $O(m \log n)$ (Task 3). The exponential algorithm is the *baseline* (path enumeration). Had we extended the problem to a *multi-criteria* or *capacity-constrained* variant, which is NP-hard, we would use pruning by the admissible bound f [2], [3], dominance pruning, and an ε-bounded weighted A* ($f = g + (1 + \varepsilon)h$) that guarantees a cost within $(1 + \varepsilon)$ of optimal, plus the per-request `time_limit_ms` returning the best route found with `truncated=true`.

Q3. Exact vs. approximate / heuristic algorithms in production?

An exact algorithm guarantees the optimal answer and a checkable certificate, at a cost that may grow quickly. A heuristic or approximate algorithm trades a bounded or unbounded loss in quality for speed [1]. Our A* is **exact** because h is admissible and consistent [2], [3]; "heuristic" describes how it is guided, not that its answer is approximate. In a rescue setting a wrong (non-optimal) route has a safety cost, so we keep the exact search and use an approximate variant only as a time-limited fallback. The hydrology model, by contrast, is an approximation (SCS-CN, a decision aid, not a hydrodynamic model) [8], and we label it as such.

Q4. Empirical benchmarking vs. asymptotic analysis?

Asymptotic analysis is done *before* implementation, on a model of computation, and states how cost grows with n, m while ignoring constants, hardware and the interpreter [1]. Empirical profiling is done *after* implementation, on real code and data, and measures constants, cache and Python overhead, and the average case that theory cannot give in closed form. We use both: Task 3 predicts $O(m \log n)$; the CI benchmark (Task 5) checks the growth trend (runtime vs. N) and enforces an absolute budget. Neither replaces the other: a good bound with a slow constant can still miss the 250 ms target, and a fast run on one dataset proves nothing about the worst case.

Q5. Cold start / data sparsity (if ML or preference matrices are used)?

The routing engine uses no learned model or user-preference matrix, so the classic cold-start problem does not apply. The analogous risk is **sparse live data**: few sensors and reports at launch, so many edges have no measured depth. Mitigation: default unknown edges to the hydrology engine's modelled depth [8], treat sensor and verified-report values as overrides, and surface edges without recent data as lower-confidence rather than silently safe. If we later add a learned depth predictor, we would seed it from the physical model and historic flood records.

Q6. How are edge-case inputs handled without crashing the service?

Every degenerate input has a defined return value rather than an exception: unknown nodes → `INVALID_NODE`; `s = t` → zero-length path; **disconnected subgraphs** or fully flooded corridors → the frontier empties and the search returns `NO_SAFE_ROUTE` (the termination certificate in line 18); **cycles** are harmless because `settled` makes every node expand once [1]; the speed floor removes division by zero. No singular-matrix case arises, as the engine uses no linear algebra. Inputs are validated at the gateway (Pydantic, bounded ranges) and errors are returned as RFC 9457 problem documents [9], so a bad request cannot take down the worker.

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
2. P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968. (Seminal source, pre-2021.)
3. R. Dechter and J. Pearl, "Generalized best-first search strategies and the optimality of A*," *J. ACM*, vol. 32, no. 3, pp. 505–536, 1985. (Seminal source, pre-2021.)
4. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. (Seminal source, pre-2021.)
5. H. Bast *et al.*, "Route planning in transportation networks," in *Algorithm Engineering*, LNCS vol. 9220. Cham, Switzerland: Springer, 2016, pp. 19–80.
6. S. Tsang and D. M. Scott, "An integrated approach to modeling the impact of floods on emergency services: A case study of Calgary, Alberta," *J. Transport Geography*, vol. 86, 2020.
7. M. Pregnolato, A. Ford, S. M. Wilkinson, and R. J. Dawson, "The impact of flooding on road transport: A depth-disruption function," *Transp. Res. D, Transp. Environ.*, vol. 55, pp. 67–81, 2017.
8. USDA Natural Resources Conservation Service, "Estimation of direct runoff from storm rainfall," in *National Engineering Handbook, Part 630 Hydrology*, ch. 10, 2004.
9. M. Nottingham, E. Wilde, and S. Dalal, "Problem details for HTTP APIs," IETF, RFC 9457, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9457
10. R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," IETF, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110
11. OpenAPI Initiative, "OpenAPI Specification v3.1.0," Feb. 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0
12. S. Ramírez, "FastAPI," software documentation. [Online]. Available: https://fastapi.tiangolo.com (accessed Nov. 2026).
13. Python Software Foundation, "heapq — Heap queue algorithm," *The Python Standard Library*, Python 3 docs. [Online]. Available: https://docs.python.org/3/library/heapq.html (accessed Nov. 2026).
14. PostGIS Project, "PostGIS manual: ST_DWithin." [Online]. Available: https://postgis.net/docs/ST_DWithin.html (accessed Nov. 2026).
15. Celery Project, "Celery: Distributed task queue," documentation. [Online]. Available: https://docs.celeryq.dev (accessed Nov. 2026). (Proposed worker queue; not yet deployed.)

MetroFlow: Flood-Aware Rescue Routing

PBL Milestone 1 · ALGOCOM / SOFTENG · Team F4 · III-CCSAD · Nov 4, 2026

Project	MetroFlow
Team	F4 · Section III-CCSAD
Members & roles	Lapating, Khent U. (Lead Algorithm Architect) · De Vera, Josiah Ray S. (Backend / Systems) · Vergara, John Gabriel D. (Data / QA) · Felipe, Marck Aaron V. (Systems Analyst / Tech Lead)
Repository	https://github.com/siahray/MetroFlow.git

The problem

- Typhoon flooding makes roads impassable **for some vehicles, not others**.
- A 28 cm unit and a 70 cm truck face different road networks.
- Dispatchers need: the **fastest route a given vehicle can actually drive**, from live water depths, or a clear “no safe route”.
- Depth-based road disruption and flood-aware emergency access are documented [6], [7].

Mathematical formulation

- **Input:** $G = (V, E)$; per edge ℓ_e (m), s_e (km/h), d_e (cm); query (s, t, c)
- **Constraint:** \(d_e
- **Objective:** minimise travel time
- **Output:** $(\text{status}, P^*, T^*)$

$$v_e = \max\left(\frac{s_e/3.6}{1+(d_e/10)^{1.8}},\ 0.5\right)$$

$$\min_{P:\ s \to t \text{ in } G_c} \sum_{e \in P} \frac{\ell_e}{v_e}$$

Depths from SCS-CN runoff [8].

Algorithm: clearance-constrained A*

Field	Detail
Paradigm	Graph theory: best-first search, min-heap [1], [2]
Baseline	Exhaustive path enumeration $O(n!)$; Dijkstra ($h \equiv 0$) [4]
Recurrence	$g(v) = \min_{(u,v)\in E_c}(g(u) + w_{uv})$, $f = g + h$
Heuristic	$h(v) = \mathrm{hav}(v, t)/v_{\max}$: **admissible and consistent** ⇒ exact, each node settled once [3]

Pseudocode (core loop)

```
g[s]=0; PQ={(h(s),0,s)}; settled={}
while PQ not empty:
    (f, gu, u) = pop_min(PQ)
    if u in settled or gu > g[u]: continue
    settled.add(u)
    if u == t: return path, g[t]
    for (v, len, speed, depth) in G[u]:
        if depth >= c: continue                  # clearance cut-off
        pen   = 1 + (depth/10)^1.8
        v_eff = max((speed/3.6)/pen, 0.5)
        g2    = gu + len / v_eff
        if g2 < g[v]: g[v]=g2; parent[v]=u; push(PQ,(g2+h(v), g2, v))
return NO_SAFE_ROUTE
```

Complexity

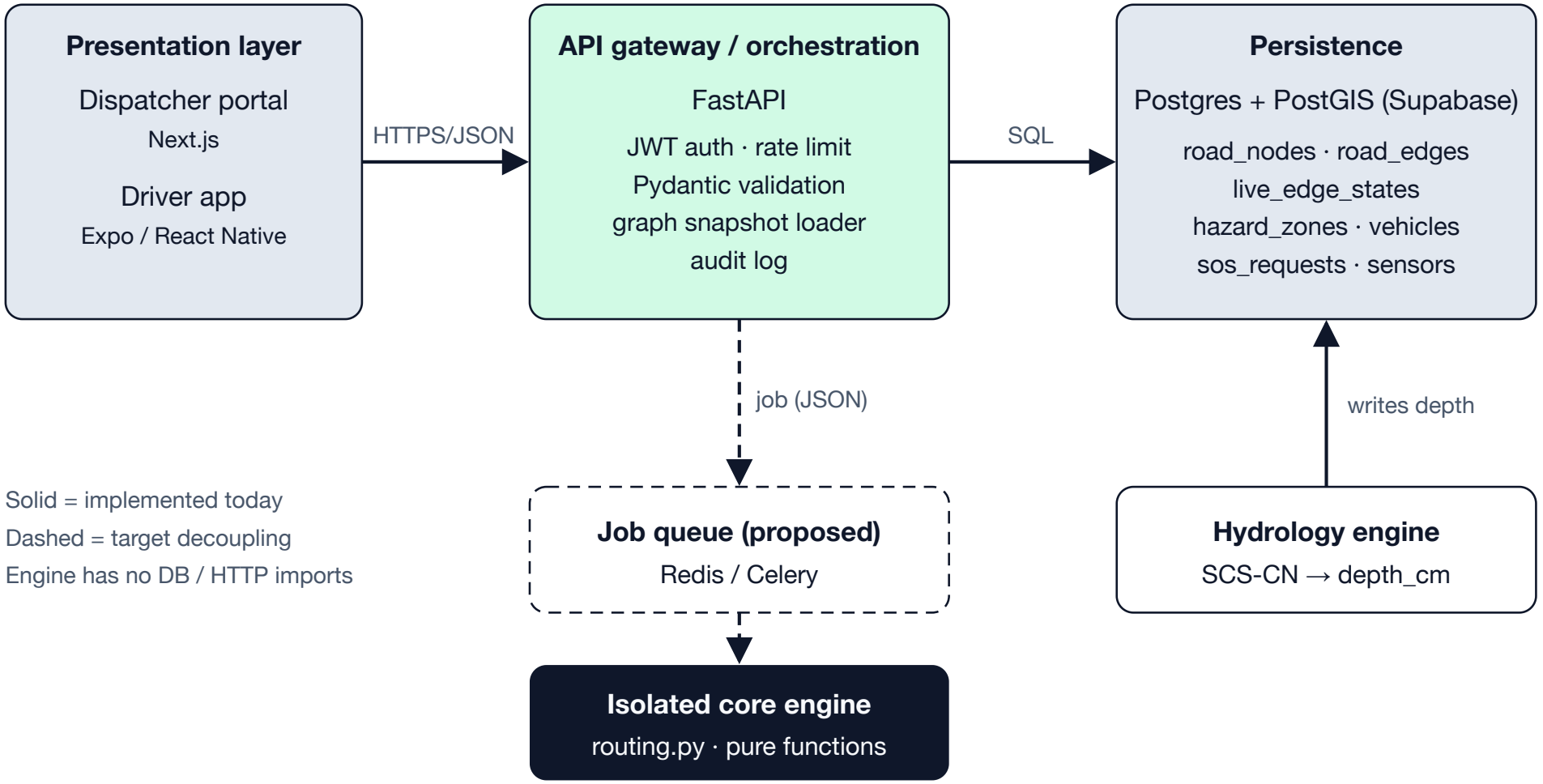
Measure	Result
Worst-case time	$O(m \log n)$: $\leq m + 1$ heap ops, $\log m \leq 2 \log n$ [1]
Best case	$\Omega(k \log n)$ when h is near-perfect (path length k)
Average	Sparse planar graph: settles a corridor, not all of V; measured in CI
Space	$O(n + m)$: g, parent, settled $O(n)$; lazy heap $\leq m + 1$
Baseline	$O(n!)$ enumeration

Master theorem not applicable: iterative, no divide-and-conquer recurrence.

Scalability (3 V's)

V	Parameter	Mitigation
Volume	Pilot district graph $\rightarrow \sim 10^5$ nodes planned	Cache topology, load by bounding box (PostGIS) [14], CSR if needed
Velocity	Typhoon bursts; K searches per SOS	Worker pool + job queue; immutable depth snapshot
Variety	GeoJSON, sensors, reports, hazards	Normalise to `cm`, `km/h`, `m` at gateway; Pydantic validation

Decoupled architecture



Data contract: `POST /api/v1/algorithm/solve`

```
// request
{ "request_id": "uuid",
  "algorithm": "td_a_star",
  "start_node": 101, "target_node": 482,
  "vehicle": { "vehicle_id": 7,
               "clearance_cm": 28.0 },
  "options": { "ignore_hazards": false,
               "time_limit_ms": 250 } }
```

```
// response certificate
{ "status": "OK",   // NO_SAFE_ROUTE | TIMEOUT
  "result": { "path": [101,134,219,482],
              "travel_time_s": 412.6,
              "distance_m": 3180.4 },
  "metrics": { "nodes_settled": 348,
               "runtime_ms": 14.2,
               "optimal": true } }
```

OpenAPI 3.1 [11]; errors as RFC 9457 problem+json [9]; HTTP semantics per RFC 9110 [10].

Agile backlog & benchmarks

- **Story 1:** Dispatcher runs per-vehicle searches to assign the lowest safe travel time.
- **Story 2:** Driver is rerouted on latest depths, or told there is no safe route.
- **DEV-ALGO-101** (`backend/routing.py`): edge cases – unknown node, `s=t`, disconnected, depth = clearance, bad speed.
- **CI assertions:** A* cost = Dijkstra cost on 10,000 random cases; p95 < 250 ms at N = 5,000 (target); A* settles ≤ Dijkstra.

Defense highlights

- **Why decouple?** CPU-bound search starves request threads; pure function is fuzz-testable; immutable snapshot keeps optimality valid.
- **Exact vs heuristic:** our A* is exact (admissible + consistent); weighted A* only as a time-limited fallback.
- **Theory vs benchmark:** Big-O predicts growth; CI measures constants and the average case.
- **Sparse data, not cold start:** default to modelled depth [8]; mark stale edges low-confidence.
- **Edge cases:** defined statuses, never exceptions; cycles harmless via `settled`.

References

1. Cormen *et al.*, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
2. Hart, Nilsson, Raphael, IEEE Trans. SSC, 4(2), 1968.
3. Dechter, Pearl, J. ACM, 32(3), 1985.
4. Dijkstra, Numer. Math., 1, 1959.
5. Bast *et al.*, Route planning in transportation networks, LNCS 9220, 2016.
6. Tsang, Scott, J. Transport Geography, 86, 2020.
7. Pregnolato *et al.*, Transp. Res. D, 55, 2017.
8. USDA NRCS, NEH Part 630, Ch. 10, 2004.
9. RFC 9457, Problem Details for HTTP APIs, 2023.
10. RFC 9110, HTTP Semantics, 2022.
11. OpenAPI Specification v3.1.0, 2021.
12. FastAPI documentation.
13. Python `heapq` documentation.
14. PostGIS manual, ST_DWithin.
15. Celery documentation.